

CS 6013: Programming Assignment

subruk

Aug 2025

Problem Statement 1

We want to compare numbers read from number plates. Each element that we want to compare looks like “CCNNNN” where C is a character from A-Z and N is a digit from 0 to 9.

For example: TS9548

We define the following total order on such numbers. Consider two distinct numbers $A = A_1A_2M_1M_2M_3M_4$ and $B = B_1B_2N_1N_2N_3N_4$.

We say $A < B$ if either:

- $A_1A_2 < B_1B_2$ in alphabetic order.
- $A_1A_2 = B_1B_2$ and $M_1M_2M_3M_4 < N_1N_2N_3N_4$ as integers.

Examples:

- TS5480 < WB1915
- AP9540 < TS7480
- KL1452 < KL1457

Input Format:

- Each line of the input will give two distinct numbers A and B separated by a space.
- Each line ends with a ‘\n’ character.
- End of input is indicated by EOF character. (You can simulate this character on linux terminals using Ctrl+D)

Output: For each line read,

- Output 1 if $A < B$ followed by a ‘\n’
- Output 0 if $A \not< B$ followed by a ‘\n’

Then read the next input line and repeat the above. Your program should end only when you read the EOF character.

Remarks:

- Create a separate function that takes two number plates and returns the comparison.
- Avoid using global variables.
- Use the input and output file provided to test your program.
- Related files (one test case and input module) are available [here](#).

Problem Statement 2:

Consider an intersection with several vehicles passing by everyday. We would like to store the number plates of all the vehicles that we see, and search for specific vehicle numbers in the future. Intuitively, the numbers that we see on this road will look quite random. So using a Binary Search Tree to store this set will suffice. Each number looks like the strings defined in Assignment 0. Construct and maintain a set of such numbers using a Binary Search Tree (BST) as the data structure and support searching for a number in this set.

Input Format:

- The first line of the input will give a list of distinct number plates $a_1, a_2, \dots, a_n$ with each a_i separated by a space. This line ends with a '\n' character.
- Each of the next lines looks like "S m \n" where m is a number plate string. (S stands for Search)
- End of input is indicated by EOF character.

Output:

No output after reading the first line. Simply create a BST with the numbers from the first line.

Once you read "S m ":

- Output 1 if m exists in the list, followed by a space and a string consisting of L's and R's. The string of L's and R's is a path from root to the node containing m where 'L' and 'R' indicate left child and right child respectively. (Assume path from root to root is the empty string). End this output line with a '\n'.
- Output 0 otherwise followed by a '\n'.

Read the next input line and repeat the above. Your program should terminate only when you read the EOF character.

Implementation Rules:

- You have to insert $a_1, a_2, \dots, a_n$ into an empty BST in the same order.
- Searching for m in the set has to be done in the BST constructed.
- Each node of your BST should contain a pointer to the two children, and parent.
- Comparison of the elements has to be done using your own function from Problem statement 1.

Remarks:

- You are encouraged to use your own code from your previous assignment submissions.

- See the public test cases to understand the i/o format.
- To check that you are building your BST correctly, it might be a good idea to use the inorder traversal as a debug string. Don't forget to remove all debug string outputs before submitting your code!
- Related files (one test case and input module) are available [here](#).